

製造業向け欠陥検出AI SaaS

基本設計書 / システム仕様書（追加整理版）

文書名：製造業向け欠陥検出AI SaaS 基本設計書

版数：v1.1

作成日：2026-03-24

作成者：申載沅

1. システム概要

本システムは、製造現場における検査画像を入力とし、CNN ベースの機械学習モデルにより製品の正常・欠陥を判定する **欠陥検出AIシステム** である。

また、本システムは単体の推論APIとしてではなく、ユーザー認証、推論履歴管理、ログ保存、管理者レビュー機能を備えた **SaaS 型の欠陥検査支援システム** として設計する。

本プロジェクトでは、欠陥データが少ない実運用上の特性を踏まえ、単に Accuracy を追求するのではなく、**欠陥見逃し（False Negative）の抑制** を最重要方針とする。

そのため、モデル評価においては **Recall（再現率）を主要指標** とし、システム全体としても「信頼できる検査支援基盤」を構築することを目的とする。

2. 背景

製造業における外観検査・品質検査では、不良品を正常品として通過させることが、品質事故、出荷不良、顧客クレーム、回収コスト、安全上の問題などにつながる重大なリスクとなる。

一方で、実際の検査データは正常サンプルに対して欠陥サンプルが極端に少ないことが多く、データセットには深刻なクラス不均衡が存在する。

このような条件下では、モデルが大半を正常と予測するだけでも高い Accuracy を示すことがあり、Accuracy のみを根拠にモデルの信頼性を判断することはできない。

そのため、本プロジェクトでは以下の観点からシステムを再設計する。

- 評価指標を Accuracy 中心から Recall 重視へ転換する
- Notebook 上の実験結果を API / DB / UI を備えた形へ発展させる
- 推論結果を記録し、誤分類や低信頼ケースを継続的に分析できる構造とする
- 将来的には外部AI支援機能（MCP連携）により、レビュー支援や類似事例分析へ拡張可能とする

3. 目的

本システムの目的は以下の通りとする。

1. 製造検査画像に対して欠陥の有無を自動判定すること
2. 深刻なクラス不均衡下でも信頼できる評価戦略を採用すること
3. 欠陥見逃しを最小化するため、Recall を主要評価指標とすること
4. 機械学習モデルを FastAPI ベースの推論 API として提供すること
5. 認証・履歴・ログ・レビュー機能を備えた SaaS 構造へ拡張すること
6. 将来的な MCP 連携により、低信頼ケースの補助分析や運用レポート生成へ発展可能な基盤を整備すること

4. 適用範囲

4.1 対象範囲

本システムの対象範囲は以下とする。

- 検査画像アップロード
- 画像前処理
- 学習済み CNN モデルによる推論
- 推論結果の API 応答
- 推論履歴の保存
- ユーザー認証
- ユーザー別の利用履歴管理
- リクエストログ / エラーログ保存
- 管理者向けレビュー対象管理
- 将来的な低信頼ケース分析支援

4.2 対象外

本設計時点では以下は対象外とする。

- 自動再学習の完全実装
- 欠陥領域の位置推定（物体検出 / セグメンテーション）
- 多クラス欠陥分類
- 本番向け高可用性クラスター構成
- 完全な MLOps パイプライン自動化

5. 前提条件

- 入力画像は、学習データと同一または近似したドメインであること
- 判定対象は二値分類（normal / defect）とする

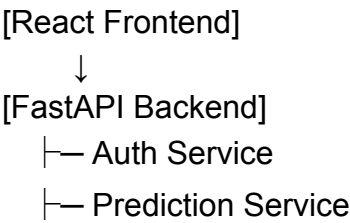
- 推論用モデルは学習済みモデルを使用する
- バックエンドは FastAPI により実装する
- フロントエンドは React を使用する
- 推論結果および関連ログは MySQL に保存する
- 今後の Docker 化、AWS 配備、MCP 連携を考慮した構成とする

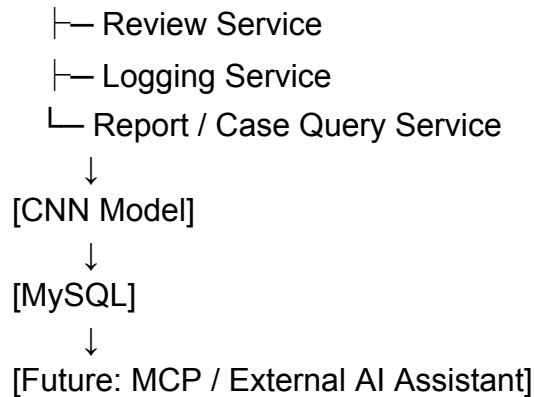
6. システム構成

6.1 全体構成

区分	構成要素	内容
Frontend	React (Vite)	画像アップロード、結果表示、履歴表示
Backend	FastAPI	認証、推論API、ログ管理、レビュー機能
ML Model	TensorFlow / AutoKeras	欠陥検出推論
Database	MySQL	ユーザー、画像、予測、ログ、レビュー情報保存
Infra	Docker / AWS	コンテナ実行、配備基盤
Future Extension	MCP連携	外部AIによる補助分析・レポート生成

6.2 システム構成図





7. 業務要件

7.1 基本業務要件

- 利用者は画像をアップロードして欠陥判定を実行できること
- システムは予測ラベルと信頼度を返却できること
- 推論結果は履歴として保存されること
- 利用者ごとにアップロード履歴を参照できること
- 管理者は低信頼ケースや要レビューケースを確認できること

7.2 品質要件

- 欠陥見逃しを最小化する評価方針であること
- 推論結果の根拠となるログを保持できること
- 誤分類・失敗ケースを分析可能であること
- 将来的な改善に向けてデータ候補を蓄積可能であること

8. データセット概要

本プロジェクトで使用するデータセットは正常画像と欠陥画像で構成される。
データ分布は以下の通りである。

分割 正常 欠陥

訓練 1102 59

テスト 276 15

8.1 データ上の課題

- 欠陥データ数が少ない
- クラス不均衡が極めて大きい
- Accuracy のみでは実運用リスクを表現できない
- Train / Test 間に類似サンプルが存在する可能性がある

このため、本システムでは Recall を主要指標とし、Precision、F1 Score、Confusion Matrix、ROC Curve を併用する。

9. モデル設計

9.1 モデル概要

項目	内容
モデル種別	CNN Classifier
実装方式	AutoKeras ImageClassifier
フレームワーク	TensorFlow / AutoKeras

9.2 学習設定

項目	内容
----	----

Loss Function	Binary Cross Entropy
---------------	----------------------

Validation Split	0.2
------------------	-----

Epochs	3
--------	---

9.3 前処理

項目	内容
----	----

リサイズ	256 × 256
------	-----------

正規化	[0,1]
-----	-------

色変換	RGB
-----	-----

9.4 ラベル定義

値 意味

0	normal
---	--------

1	defect
---	--------

10. 評価方針

10.1 評価戦略

本システムでは、製造工程において最も重大な失敗が **欠陥の見逃し** であるため、Recall を主要評価指標とする。

Accuracy は補助指標として扱い、単独ではモデル採用判断に用いない。

10.2 評価指標

- Accuracy
- Precision

- Recall (主要指標)
- F1 Score
- Confusion Matrix
- ROC Curve

10.3 評価上の留意事項

以下の理由により、現時点の評価結果は楽観的に見える可能性がある。

- データセット規模が小さい
- 欠陥クラスが極端に少ない
- 画像分布の偏りがある可能性
- 外部データで未検証

今後の追加検証方針は以下の通りとする。

- Cross Validation
- 外部データセット評価
- Threshold Calibration
- Data Augmentation
- Class Weighting

11. 機能仕様

11.1 ユーザー認証機能

概要

利用者ごとに画像アップロード・履歴管理を行うため、認証機能を提供する。

機能一覧

- 会員登録

- ログイン
- JWT 発行 / 検証
- 認証済みユーザーのみアップロード可能とする

目的

- 利用者単位のデータ管理
- 利用履歴管理
- SaaS としての基本利用制御

この要件は、ロードマップ上でも SaaS 化の中核機能として整理されている。

11.2 ヘルスチェック機能

エンドポイント

Method Path 説明

GET /health API 稼働状態確認

レスポンス例

```
{  
  "status": "ok"  
}
```

11.3 欠陥判定機能

概要

アップロードされた画像に対し、前処理および CNN 推論を実行し、判定結果を返却する。

エンドポイント

Method Path 説明

POST /predict 画像を受信して推論結果を返却する

リクエスト

- Content-Type: multipart/form-data
- パラメータ: file

レスポンス例

```
{  
  "image_name": "sample.jpg",  
  "prediction": "defect",  
  "confidence": 0.91  
}
```

処理概要

1. 認証状態を確認する
2. 画像を受信する
3. 前処理を実施する
4. CNN モデルで推論する
5. 結果を保存する
6. 応答を返却する

異常系

- 未認証アクセス
- 不正ファイル形式
- 空ファイル
- 前処理失敗

- モデル読み込み失敗
- 推論失敗
- DB 保存失敗

11.4 推論履歴参照機能

概要

ユーザーは自身の過去の推論結果を参照できる。

主な用途

- 過去判定結果の確認
- 誤判定ケースの見直し
- モデル改善用ケースの抽出

11.5 ログ管理機能

概要

推論APIの利用履歴および障害分析のため、各種ログを保存する。

対象ログ

- リクエストログ
- エラーログ
- 推論結果ログ
- 管理者レビュー結果

目的

- 障害原因調査

- 失敗ケース把握
- 継続改善のためのデータ蓄積

11.6 管理者レビュー機能

概要

低信頼または誤分類疑いのケースを管理者が確認できるよう、レビュー対象管理機能を設ける。

想定機能

- review_queue への登録
- pending / reviewed / rejected 等の状態管理
- human_label 記録
- dataset candidate への昇格管理

この構造は、推論システムを継続改善型へ発展させる上で重要である。

12. 推論フロー

ユーザーがログイン



画像アップロード



POST /predict



画像前処理



CNN 推論



推論結果生成

↓
predictions / logs / review候補保存
↓
レスポンス返却

13. DB設計

13.1 設計方針

本システムでは、単なる prediction 保存にとどまらず、SaaS としての利用管理、分析、レビュー、将来の再学習候補管理まで考慮した DB 設計を行う。

13.2 テーブル一覧

テーブル名	用途
users	ユーザー管理
uploaded_images	アップロード画像管理
predictions	推論結果管理
request_logs	API リクエストログ
error_logs	エラーログ
review_queue	要レビューケース管理
dataset_candidates	再学習候補データ管理

13.3 主要テーブル定義

users

カラム名	内容
id	主キー
email	ログインID
password_hash	ハッシュ化パスワード
role	user / admin
created_at	作成日時

predictions

カラム名	内容
id	主キー
image_id	対象画像ID
user_id	実行ユーザーID
label	予測ラベル
confidence	推論信頼度
status	推論状態
created_at	登録日時

review_queue

カラム名	内容
id	主キー
prediction_id	対象予測ID
review_status	pending / reviewed / rejected

カラム名	内容
------	----

human_label	人手判定ラベル
-------------	---------

created_at	登録日時
------------	------

13.4 拡張フィールド候補

将来的な AI 補助分析を見据え、以下のような項目を追加可能とする。

- review_status
- human_label
- ai_review_summary
- ai_review_recommendation
- is_low_confidence
- is_dataset_candidate
- failure_reason
- similar_case_group

14. フロントエンド仕様

14.1 概要

フロントエンドは React により実装し、ユーザーがブラウザから直感的に画像アップロードおよび結果確認を行えるようにする。

14.2 主な機能

- ログイン画面
- 画像アップロード画面
- 推論結果表示
- ユーザー別履歴表示

- 管理者向けレビュー対象表示
- 将来的な AI 分析ボタン追加

14.3 画面フロー

1. ユーザーがログイン
2. 画像を選択
3. Upload 実行
4. 推論結果を表示
5. 必要に応じて履歴またはレビュー画面で確認

15. 非機能要件

15.1 信頼性

- 欠陥見逃しを抑制する評価方針を採用すること
- 推論・ログ・レビュー結果を保存すること
- 障害発生時に追跡可能であること

15.2 保守性

- Router / Service / DB 処理を分離すること
- PredictionService / ReviewService / CaseQueryService / ReportService を段階的に分割可能とすること
- 設定値を外部化すること

15.3 拡張性

- JWT 認証追加に対応すること

- 管理者レビュー機能を追加できること
- Docker 化および AWS 配備に対応すること
- MCP 連携のための API / DB 拡張が可能であること

15.4 運用性

- ログ確認手順を整理できること
- 障害確認順序を文書化できること
- デプロイ手順を README 等に残せること

これらはロードマップにおける「API 構造整理」「Docker / AWS」「設計理由説明可能化」と整合する。

16. インフラ・配備方針

16.1 ローカル実行

- API コンテナ化
- MySQL コンテナ化
- docker-compose 等による一括起動

16.2 本番配備

- AWS (例: EC2) への配備を想定
- 環境変数管理
- 開発 / 本番環境の分離
- health check 実行確認
- 実サーバー上で Upload → Predict → DB 保存までの疎通確認

16.3 保存方針

- 画像保存先は local または S3 を検討
- ログ保存先および確認方法を別途運用設計で定義

17. 今後の拡張方針

17.1 モデル面

- Data Augmentation
- Class Weighting
- Threshold 最適化
- 大規模データによる再検証
- 外部データセット評価

17.2 システム面

- 履歴一覧機能強化
- 管理者レビュー画面
- 失敗ケースの分離保存
- データセット候補管理
- キャッシュ導入
- async 処理導入

17.3 MCP / 外部AI連携

将来的には、MCP を通じて外部AIが以下を補助できる構成を想定する。

- 低信頼ケースの説明

- 過去類似事例の検索
- 管理者レビュー補助
- 運用レポート生成
- 再学習候補提案

想定される MCP Tool 例 :

- `predict_defect(image_id)`
- `get_prediction_result(case_id)`
- `list_low_confidence_cases(limit)`
- `get_case_detail(case_id)`
- `get_similar_cases(case_id)`
- `save_ai_review(case_id, summary, recommendation)`

これは現行システムの代替ではなく、**既存の欠陥検出システムを拡張する補助レイヤー**として位置づける。

18. 技術スタック

Backend

- Python
- FastAPI
- MySQL

ML / DL

- TensorFlow
- AutoKeras
- NumPy
- Pillow

Frontend

- React (Vite)
- Tailwind CSS
- JavaScript

Infra

- Docker
- AWS

Future Extension

- MCP
- External AI Assistant

19. 制約事項および課題

19.1 現時点の課題

- データセットが小規模
- クラス不均衡が大きい
- 閾値調整が未十分
- ドメイン外画像に対する汎化性能が未検証
- Notebook ベース実験から SaaS 化への移行過程にある

19.2 対応方針

- モデル評価の厳格化
- ログの蓄積と失敗分析
- DB / API / UI の段階的拡張
- 配備と運用を含めたシステム化
- 将来的な AI 補助分析基盤の追加

20. まとめ

本システムは、製造業向け欠陥検出における **Accuracy 依存の危険性** を踏まえ、**Recall 重視で信頼性を確保する欠陥検出AI SaaS** として設計するものである。

また、単なる機械学習モデル実装にとどまらず、認証、履歴管理、ログ保存、レビューキュー、将来的な MCP 連携までを見据えることで、実運用に近いシステム基盤を構築する。

すなわち本プロジェクトは、

「**CNN による欠陥検出モデルの実装**」に加え、

「**そのモデルを現実のサービスとして提供・運用・改善できるバックエンド / SaaS 構造へ発展させること**」を目的とする。